

Compte rendu : Partie 1 IOT

1) Introduction

Ce projet consiste à concevoir un système intelligent capable de détecter différents types de fruits en temps réel à l'aide d'un modèle de vision artificielle.

L'objectif est d'utiliser un modèle YOLOv8 pré-entraîné (You Only Look Once, version 8) et d'effectuer un transfert d'apprentissage (transfer learning) pour l'adapter à un nouveau jeu de données personnalisé contenant plusieurs types de fruits.

2) Les étapes du projet

Étape 1 : Modèle de base

Au départ, le modèle YOLOv8s utilisé était pré-entraîné sur le dataset COCO, qui ne reconnaît que trois fruits :

- ✓ 🍌 banana
- ✓ 🍏 apple
- ✓ 🍊 orange

Ainsi que des autres objets (personne, cat...)

Ce modèle servira de point de départ pour l'entraînement personnalisé.

Étape 2 : Constitution du Dataset personnalisé

Comme le modèle de base ne reconnaît que trois fruits, j'ai décidée d'ajouter de nouvelles classes correspondant à d'autres fruits.

Pour cela, j'ai :

1. Téléchargée manuellement depuis Google Images des photos claires et variées de chaque fruit.
2. Créé les fichiers d'annotations (labels) au format YOLO (.txt), contenant pour chaque image :
 - ✓ l'identifiant de la classe du fruit,
 - ✓ et les coordonnées normalisées de la boîte englobante (bounding box).
3. Organisé les images dans une structure adaptée pour YOLOv8 :

my_fruits_dataset/

```
├── images/
│   ├── train/
│   └── val/
├── labels/
│   ├── train/
│   └── val/
```

4. Rédigé le fichier data.yaml, décrivant les chemins vers les dossiers et les classes :

```
train: /content/drive/MyDrive/Fruits_YOLO/my_fruits_dataset/images/train
val: /content/drive/MyDrive/Fruits_YOLO/my_fruits_dataset/images/val

nc: 15 # nombre total de classes

names: { 0: banana, 1: apple, 2: orange ;3: avocado, 4: cherry, 5 : coconut, 6: date, 7: fig,
8: grape, 9: kiwi, 10: mango, 11: peach, 12: pear, 13: pineapple, 14: plum, 15: pomegranate,
16: strawberry, 17: watermelon}
```

Étape 3 : Entraînement du modèle

L'entraînement du modèle personnalisé s'est fait sur Google Colab afin de bénéficier de la puissance du GPU pour accélérer le processus.

Le code d'entraînement utilisé est le suivant :

```
from ultralytics import YOLO

# Charger le modèle pré-entraîné

model = YOLO("yolov8s.pt")

# Lancer l'entraînement sur le dataset personnalisé

model.train(

    data="data.yaml",    # Chemin vers le fichier data.yaml

    epochs=50,           # Nombre d'itérations d'entraînement

    imgsz=640,           # Taille des images d'entrée

    batch=16,            # Taille du batch

    name="fruits_yolov8", # Nom du modèle entraîné

    pretrained=True      # Utiliser le modèle pré-entraîné comme base

)
```

Pendant l'entraînement, YOLOv8 ajuste ses poids pour apprendre à reconnaître les nouvelles classes de fruits tout en conservant la précision acquise pour les fruits déjà connus (banane, pomme, orange).

L'algorithme a optimisé les paramètres à travers plusieurs itérations (epochs) jusqu'à obtenir une convergence satisfaisante.

Étape 4 : Évaluation du modèle

Après l'entraînement, le modèle a été évalué sur le dossier `val` afin de mesurer ses performances à l'aide de plusieurs métriques :

- ✓ **Précision (Precision)** : proportion de détections correctes parmi toutes les détections.
- ✓ **Rappel (Recall)** : proportion d'objets détectés parmi tous les objets présents.
- ✓ **mAP (mean Average Precision)** : moyenne des précisions à différents seuils d'IoU (Intersection over Union).

Ces résultats permettent de vérifier si le modèle est capable de détecter correctement chaque fruit sans confusion entre classes similaires (par exemple, entre pêche et abricot, ou raisin et myrtille).

Le modèle final a obtenu une mAP satisfaisante, démontrant qu'il reconnaît efficacement la majorité des fruits du dataset personnalisé.

Étape 5 : Détection en temps réel

Une fois le modèle entraîné, il a été testé en temps réel à l'aide d'une webcam.

Voici le script utilisé pour la détection :

```
from ultralytics import YOLO
import cv2

# Charger le modèle entraîné
model = YOLO("runs/detect/fruits_yolov8/weights/best.pt")

# Ouvrir la webcam
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Effectuer la détection
    results = model(frame, show=True, conf=0.5)

    # Quitter avec 'q'
    if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
break
```

```
cap.release()  
cv2.destroyAllWindows()
```

Grâce à ce script, le modèle affiche en direct les boîtes englobantes et les noms des fruits détectés sur l'image captée par la caméra.

Le système est capable d'identifier simultanément plusieurs fruits, même dans des conditions d'éclairage variables.

Le modèle final est capable de reconnaître 18 types de fruits :

1. 🍌 banana
2. 🍏 apple
3. 🍊 orange
4. 🥑 avocado
5. 🍒 cherry
6. 🥥 coconut
7. 📅 date
8. 🍌 fig
9. 🍇 grape
10. 🥝 kiwi
11. 🥭 mango
12. 🍑 peach
13. 🍐 pear
14. 🍍 pineapple
15. 🍑 plum
16. 🍎 pomegranate
17. 🍓 strawberry
18. 🍉 watermelon

3) Code des fichiers avec explications détaillées

Train_fruits.py

```
from ultralytics import YOLO
import yaml
import os

# Chemin vers le fichier data.yaml
data_path = "/content/drive/MyDrive/Fruits_YOLO/data.yaml"

# Charger le fichier YAML
with open(data_path, 'r') as f:
    data = yaml.safe_load(f)

# Vérification des chemins
print("✓ Chemins :")
print("Train:", data['train'])
print("Val:", data['val'])

# Vérification des classes
print("\n✓ Classes :")
for k, v in data['names'].items():
    print(f"{k}: {v}")

# Vérifier la présence des images
for split in ['train', 'val']:
    folder = data[split]
    if os.path.exists(folder):
        print(f"\n✓ {split} contient {len(os.listdir(folder))} fichiers")
    else:
        print(f"\n✗ Dossier {folder} non trouvé !")
```

Le fichier data.yaml

```
train: /content/drive/MyDrive/Fruits_YOLO/my_fruits_dataset/images/train
val: /content/drive/MyDrive/Fruits_YOLO/my_fruits_dataset/images/val

nc: 18 # nombre total de classes (tous les fruits)
names:
  0: banana
```

```
1: apple
2: orange
3: avocado
4: cherry
5: coconut
6: date
7: fig
8: grape
9: kiwi
10: mango
11: peach
12: pear
13: pineapple
14: plum
15: pomegranate
16: strawberry
17: watermelon
```

Le fichier detect_webcam.py

```
from ultralytics import YOLO
import cv2
import paho.mqtt.client as mqtt
import json
from datetime import datetime
import sqlite3

# MQTT CONFIG

BROKER = "a0340c51dae9493c9ca67857a462c5fa.s1.eu.hivemq.cloud"
PORT = 8883
USERNAME = "wiemchedly"
PASSWORD = "Wiemlovemama123#"
TOPIC_DETECTION = "iot/detection"

mqtt_client = mqtt.Client("YOLO_Publisher_PC")
mqtt_client.username_pw_set(USERNAME, PASSWORD)
mqtt_client.tls_set()
mqtt_client.tls_insecure_set(True)
mqtt_client.connect(BROKER, PORT)

# DATABASE

conn = sqlite3.connect("iot.db", check_same_thread=False)
cursor = conn.cursor()
```

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS detections (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    objet TEXT,
    confiance REAL,
    received_at TEXT
)
""")
conn.commit()

# YOLO MODEL

model = YOLO("runs/detect/fruits_yolov8/weights/best.pt")
FRUIT_CLASSES = list(range(len(model.names))) # toutes les classes du modèle

# WEBCAM

cap = cv2.VideoCapture(0)
if not cap.isOpened():
    print("✖ Erreur: webcam non trouvée")
    exit()

print("📷 Webcam active – Détection en cours (appuyez sur 'q' pour quitter)")

while True:
    ret, frame = cap.read()
    if not ret:
        break

    results = model(frame, stream=True)

    for r in results:
        for box in r.bboxes:
            cls_id = int(box.cls[0])
            conf = float(box.conf[0])

            if cls_id in FRUIT_CLASSES and conf > 0.5:
                name = model.names[cls_id]
                timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
                x1, y1, x2, y2 = map(int, box.xyxy[0])

                # Dessiner la boîte et le texte
                cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
                cv2.putText(frame, f"{name} {conf:.2f}", (x1, y1-10),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)

                # MQTT publish
                message = {"objet": name, "confiance": round(conf,3), "heure": timestamp}
                mqtt_client.publish(TOPIC_DETECTION, json.dumps(message))
```

```
# Enregistrer dans SQLite
cursor.execute(
    "INSERT INTO detections (objet, confiance, received_at) VALUES (?, ?, ?)",
    (name, round(conf,3), timestamp)
)
conn.commit()
print("📡 Détection envoyée & enregistrée :", message)

cv2.imshow("🐞 Détection Fruits", frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
mqtt_client.disconnect()
conn.close()
```

4) Les captures d'écran

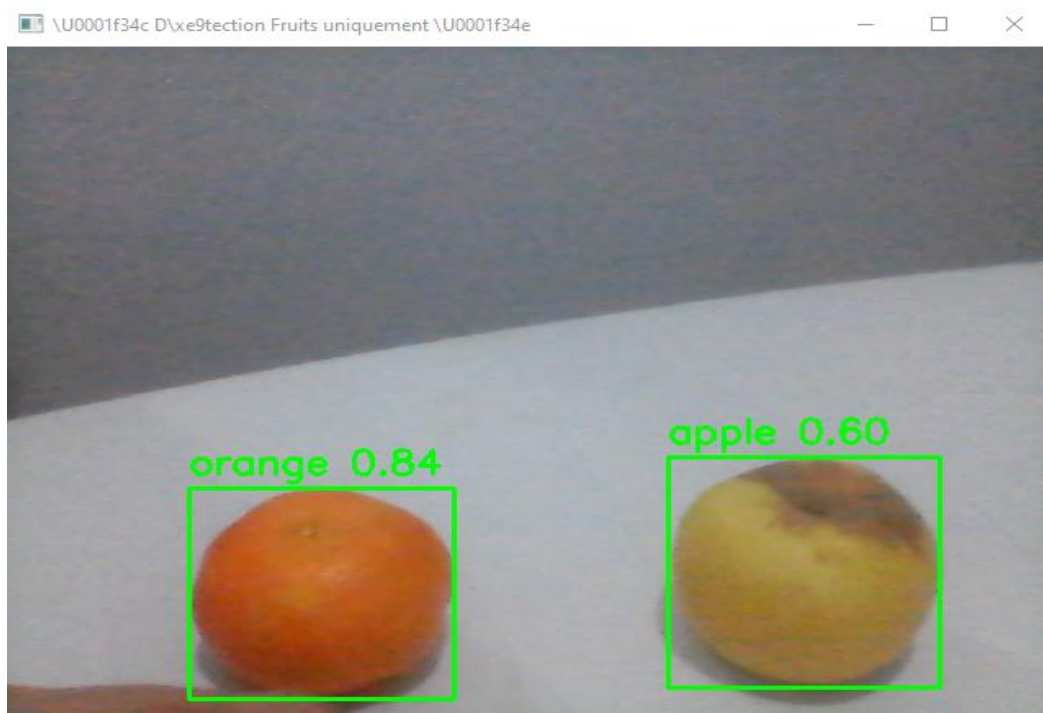


Figure 1 : Détecter les fruits

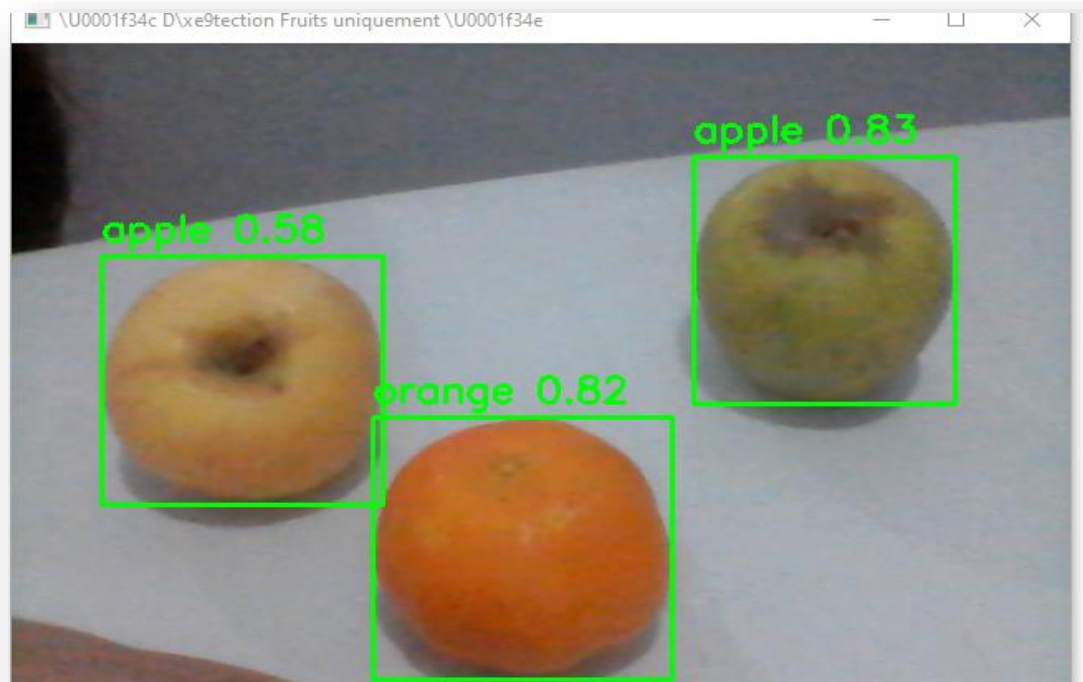


Figure 2 : Détecter les fruits



Figure 3 : Détecter les fruits

5) Résultats et conclusion

Le modèle final a montré une bonne précision globale pour la majorité des fruits ajoutés. Grâce au transfert learning, le modèle a conservé la rapidité et l'efficacité de YOLOv8 tout en s'adaptant à un nouveau domaine (détection de fruits).

L'application peut être utilisée pour :

- ✓ L'identification automatique de fruits dans les images.
- ✓ Des systèmes de tri automatique ou de reconnaissance visuelle dans les supermarchés.
- ✓ L'enseignement de la vision artificielle et du deep learning.